

```
In [5]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

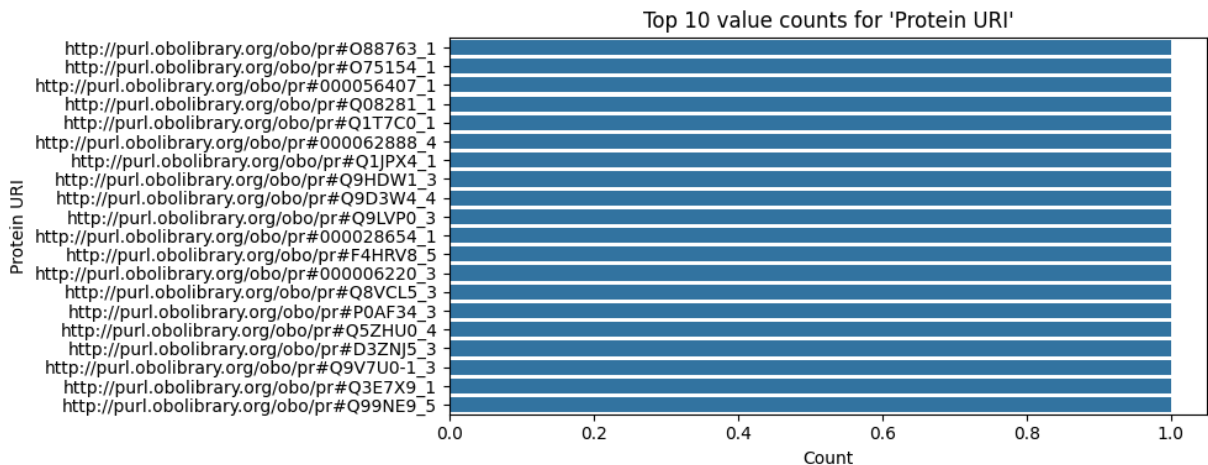
# Load CSV
df = pd.read_csv("protein_GAN_summary.csv")

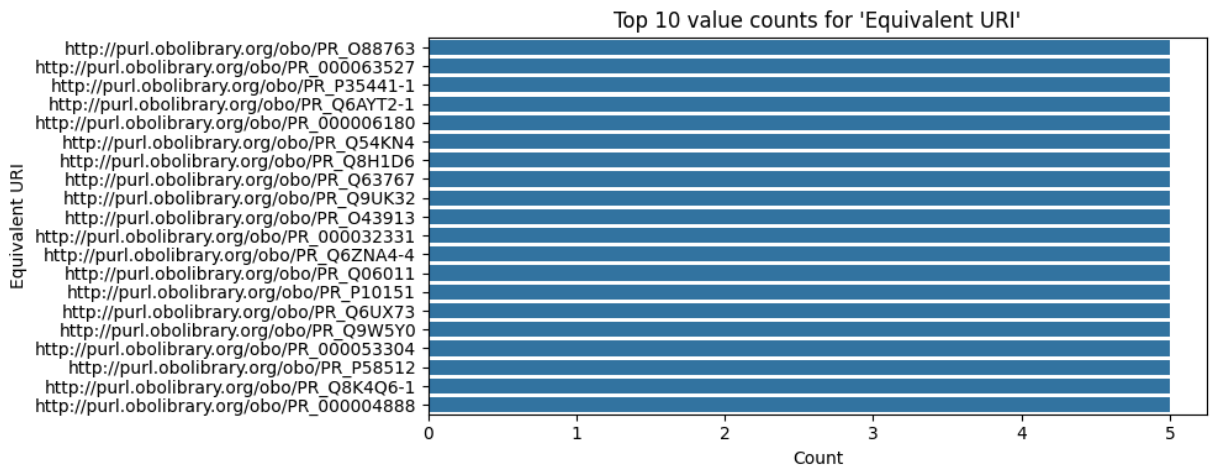
# List columns
print(df.columns)

# Function to plot distributions
def plot_distributions(df):
    for col in df.columns:
        plt.figure(figsize=(10, 4))
        if df[col].dtype == 'object':
            # Categorical/text: show top 10 frequent values
            top_values = df[col].value_counts().nlargest(20)
            sns.barplot(x=top_values.values, y=top_values.index)
            plt.title(f"Top 10 value counts for '{col}'")
            plt.xlabel("Count")
            plt.ylabel(col)
        else:
            # Numeric: histogram
            sns.histplot(df[col].dropna(), kde=True, bins=30)
            plt.title(f"Distribution of '{col}'")
        plt.tight_layout()
        plt.show()

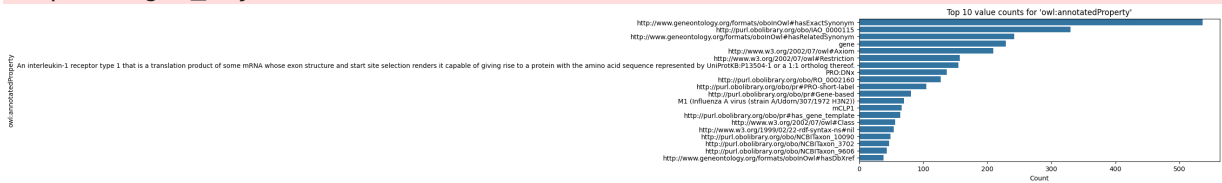
plot_distributions(df)
```

```
Index(['Protein URI', 'Equivalent URI', 'owl:annotatedProperty',
      'owl:annotatedSource', 'rdfs:comment', 'oboInOwl:hasBroadSynonym',
      'oboInOwl:hasDbXref', 'oboInOwl:hasExactSynonym',
      'oboInOwl:hasNarrowSynonym', 'oboInOwl:hasRelatedSynonym', 'rdfs:label',
      'owl:onProperty', 'owl:someValuesFrom', 'rdfs:subClassOf', 'rdf:type'],
      dtype='object')
```

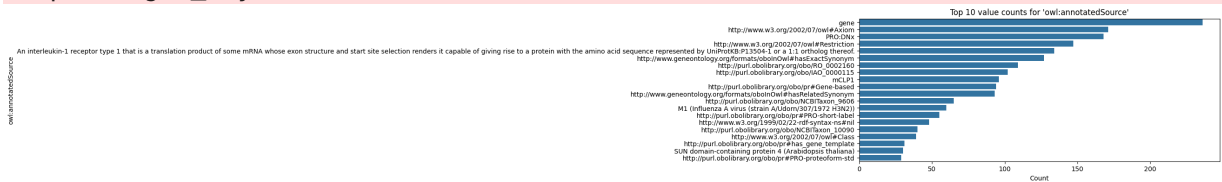




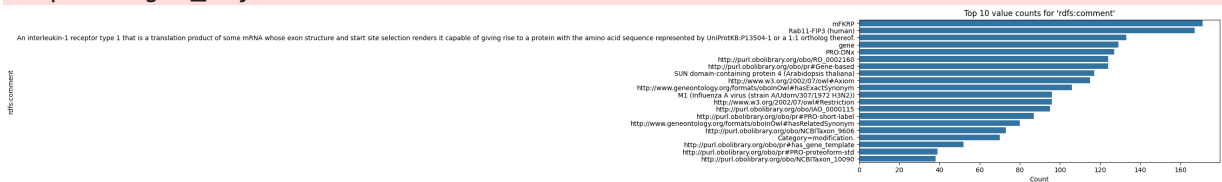
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.py:26: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.
plt.tight_layout()



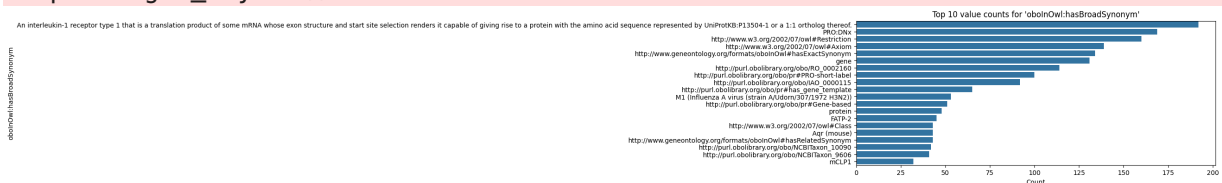
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.py:26: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.
plt.tight_layout()



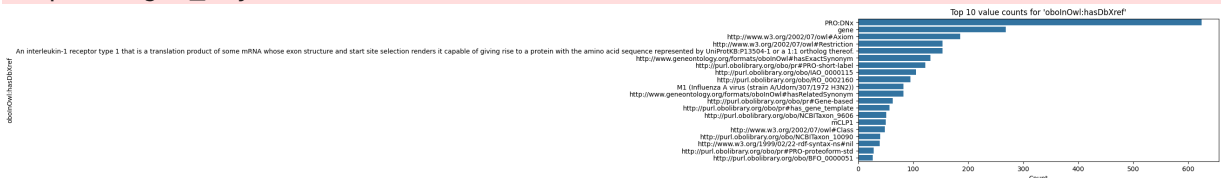
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.py:26: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.
plt.tight_layout()



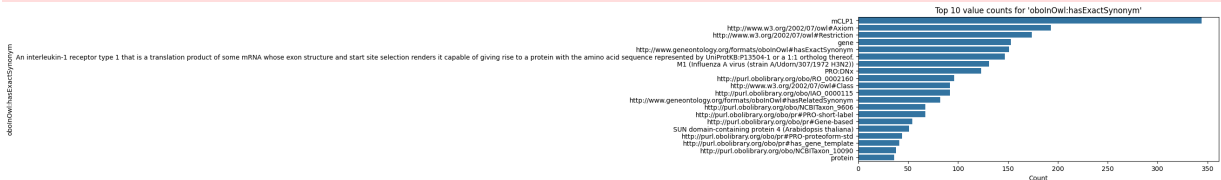
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.py:26: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.
plt.tight_layout()



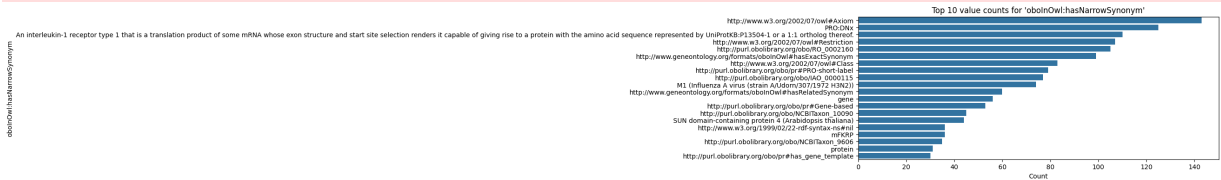
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.py:26: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.
plt.tight_layout()



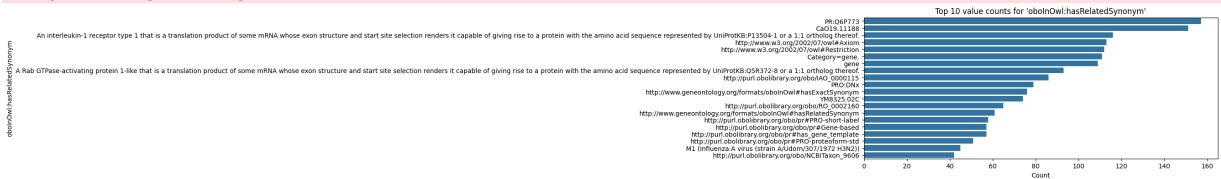
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.py:26: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.
plt.tight_layout()



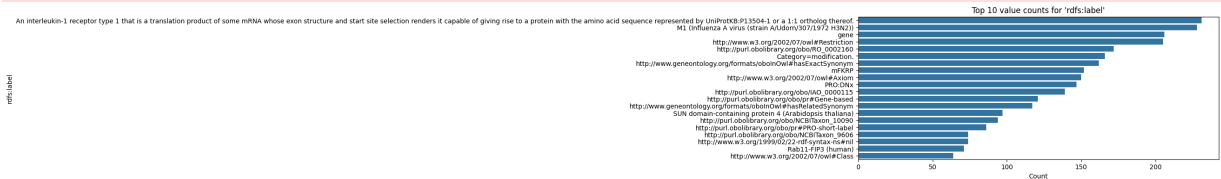
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.py:26: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.
plt.tight_layout()



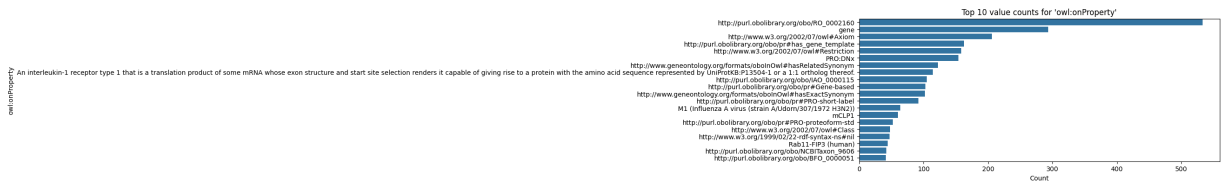
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.py:26: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.
plt.tight_layout()



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.py:26: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.
plt.tight_layout()



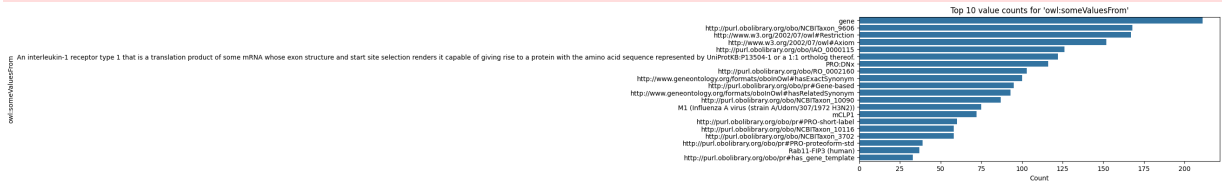
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.py:26: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.
plt.tight_layout()



```

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.
py:26: UserWarning: Tight layout not applied. The left and right margins can
not be made large enough to accommodate all Axes decorations.
plt.tight_layout()

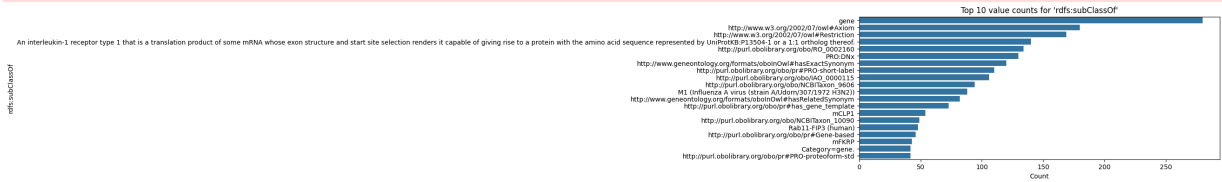
```



```

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/2002151382.
py:26: UserWarning: Tight layout not applied. The left and right margins can
not be made large enough to accommodate all Axes decorations.
plt.tight_layout()

```



```

In [22]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Set Seaborn theme and context
sns.set_theme(style="whitegrid")
sns.set_context("talk") # Larger fonts

# Load CSV
df = pd.read_csv("protein_VAE_summary.csv")

# Function to plot distributions with enhancements
def plot_distributions(df):
    for col in df.columns:
        plt.figure(figsize=(12, 5))
        if df[col].dtype == 'object':

```

```

top_values = df[col].value_counts().nlargest(30)
sns.barplot(
    x=top_values.values,
    y=top_values.index,
    palette="viridis"
)
plt.title(f"Top 10 Value Counts for '{col}'", fontsize=16, weight='bold')
plt.xlabel("Count", fontsize=14)
plt.ylabel(col, fontsize=14)

else:
    sns.histplot(
        df[col].dropna(),
        kde=True,
        bins=30,
        color="skyblue",
        edgecolor="black"
    )
    plt.title(f"Distribution of '{col}'", fontsize=16, weight='bold')
    plt.xlabel(col, fontsize=14)
    plt.ylabel("Frequency", fontsize=14)
    plt.xticks(fontsize=7)
    plt.yticks(fontsize=7)
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.show()

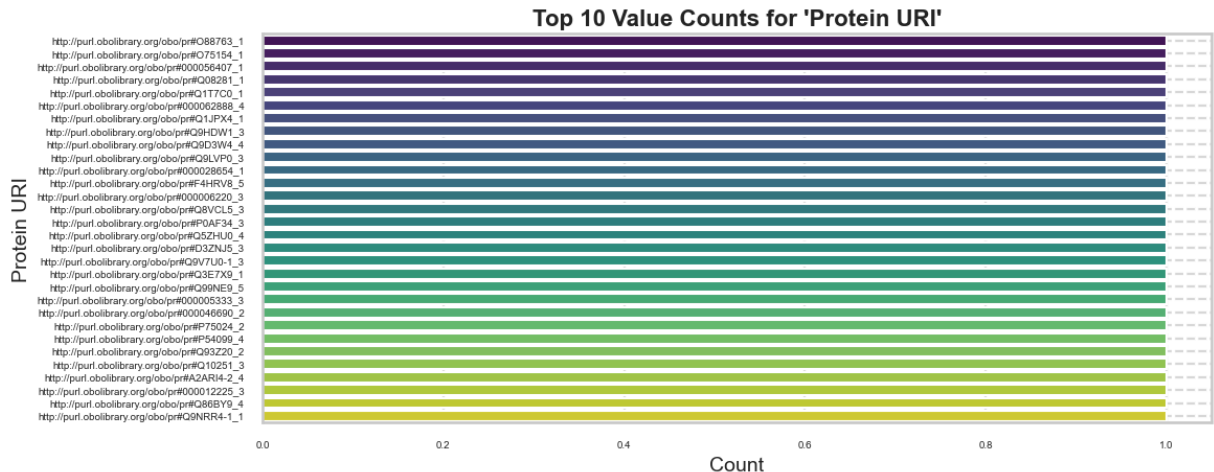
plot_distributions(df)

```

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.
py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

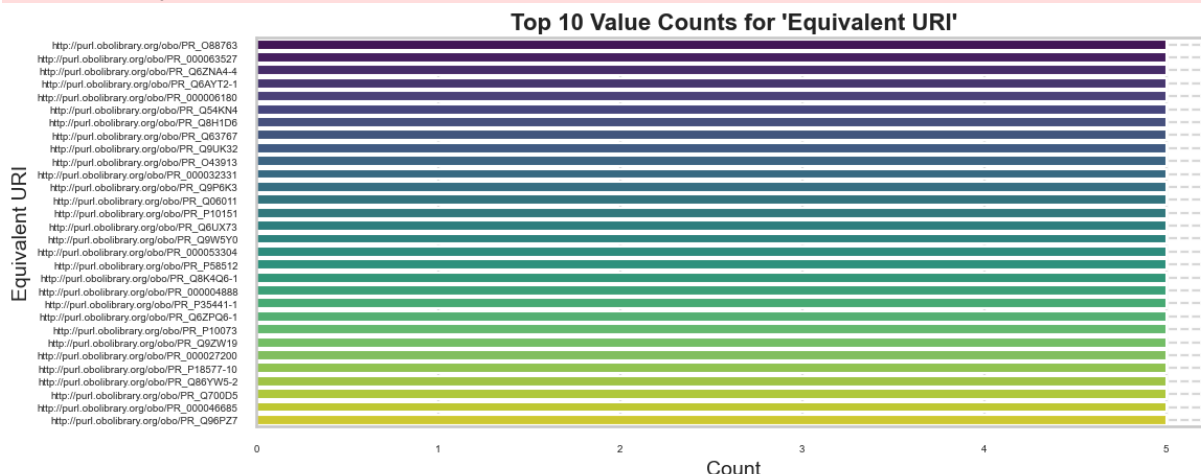
sns.barplot(



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.py:18: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

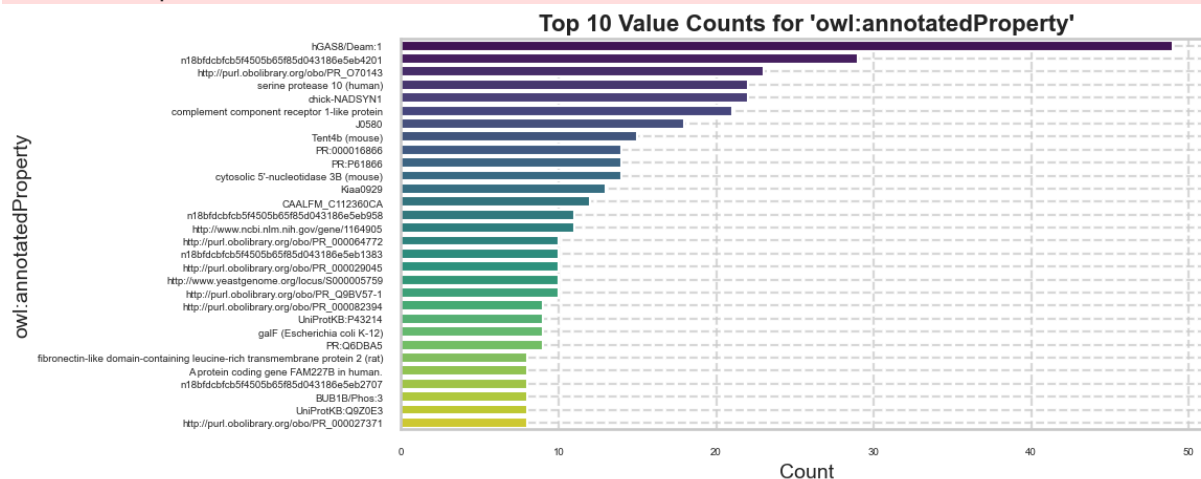
```
sns.barplot(
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.py:18: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

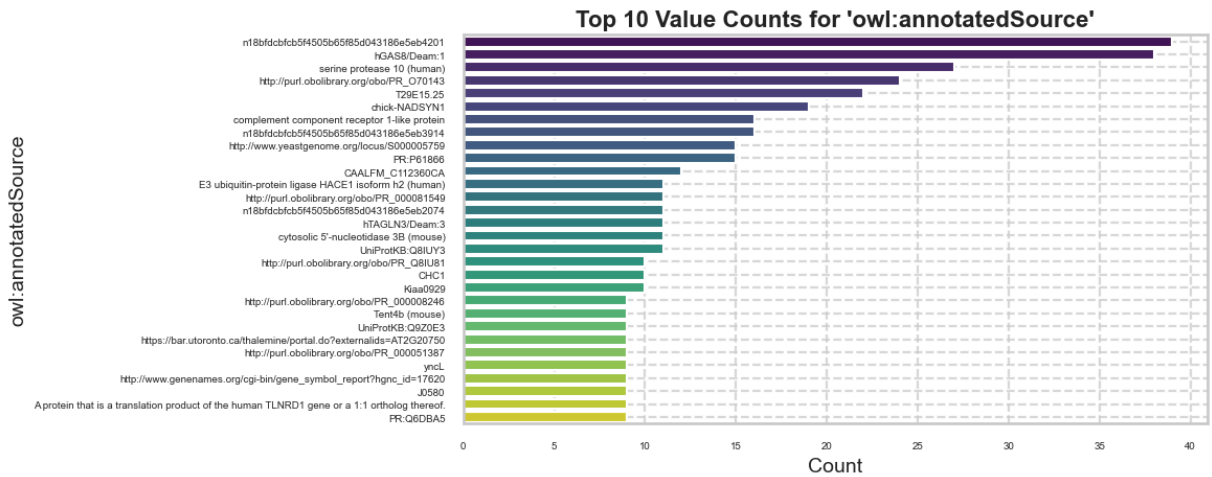
```
sns.barplot(
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.py:18: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.
py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

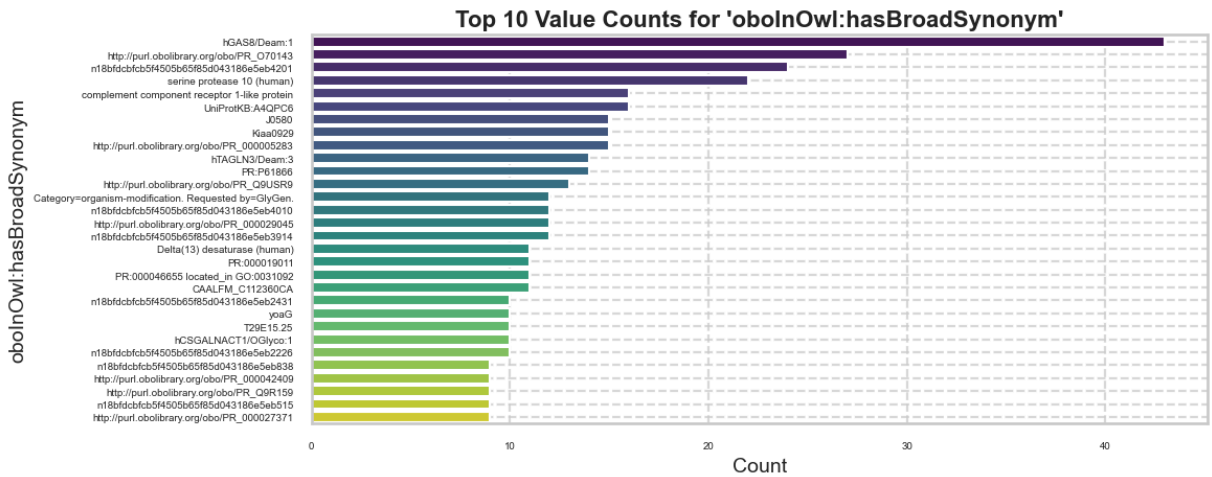
sns.barplot(



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.
py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

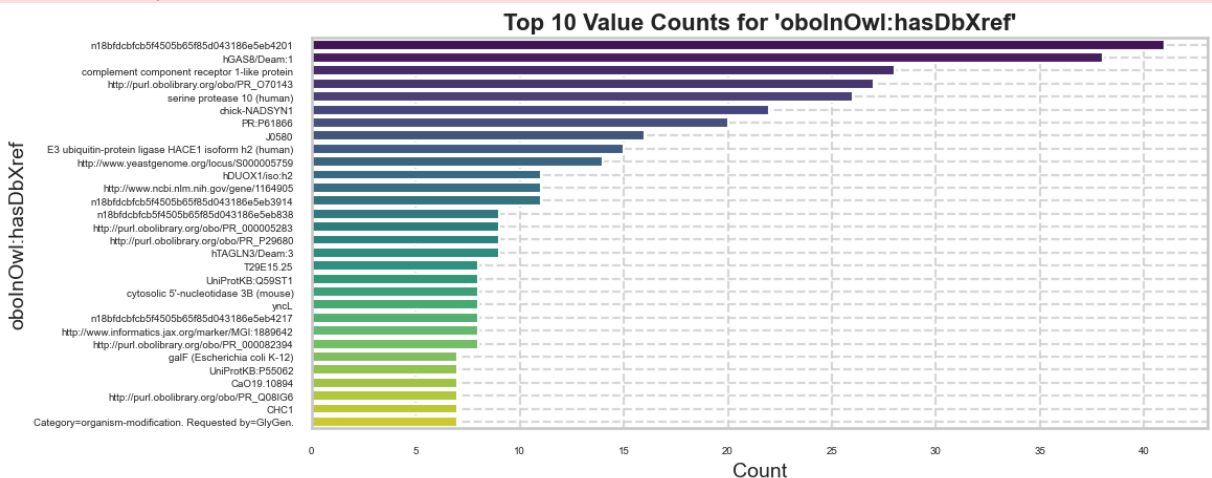
sns.barplot(



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.
py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

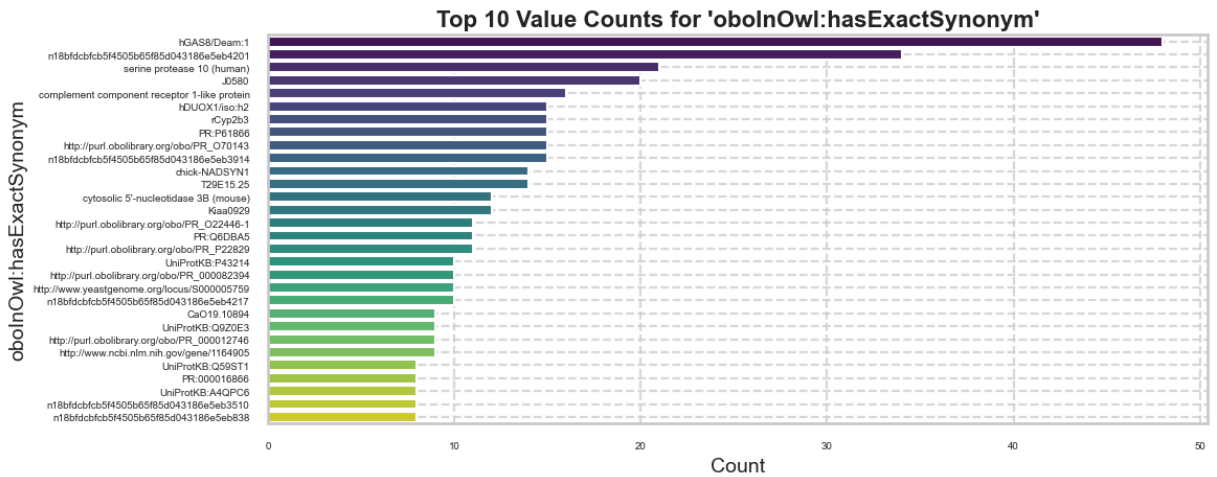
sns.barplot(



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.
py:18: FutureWarning:

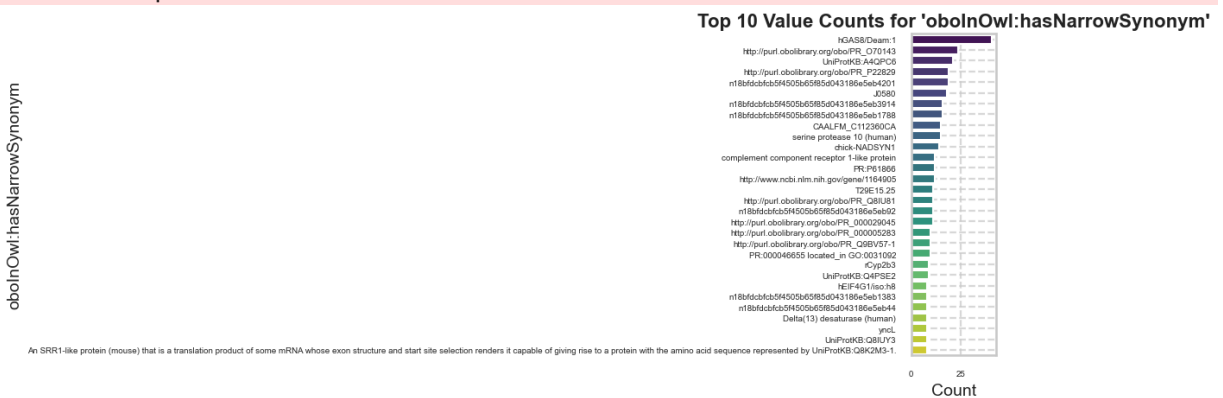
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

sns.barplot(



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.py:18: FutureWarning:
```

Passing ``palette`` without assigning ``hue`` is deprecated and will be removed in v0.14.0. Assign the ``y`` variable to ``hue`` and set ``legend=False`` for the same effect.



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.  
py:18: FutureWarning:
```

Passing ``palette`` without assigning ``hue`` is deprecated and will be removed in v0.14.0. Assign the ``y`` variable to ``hue`` and set ``legend=False`` for the same effect.

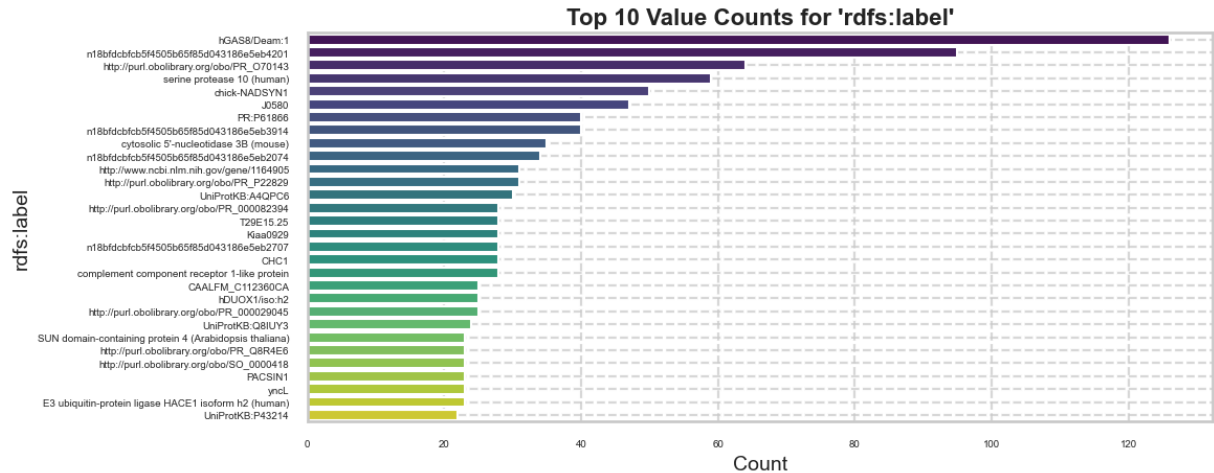
```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf400000gn/T/ipykernel_56106/3092666243.py:40: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.  
    plt.tight_layout()
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.py:18: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

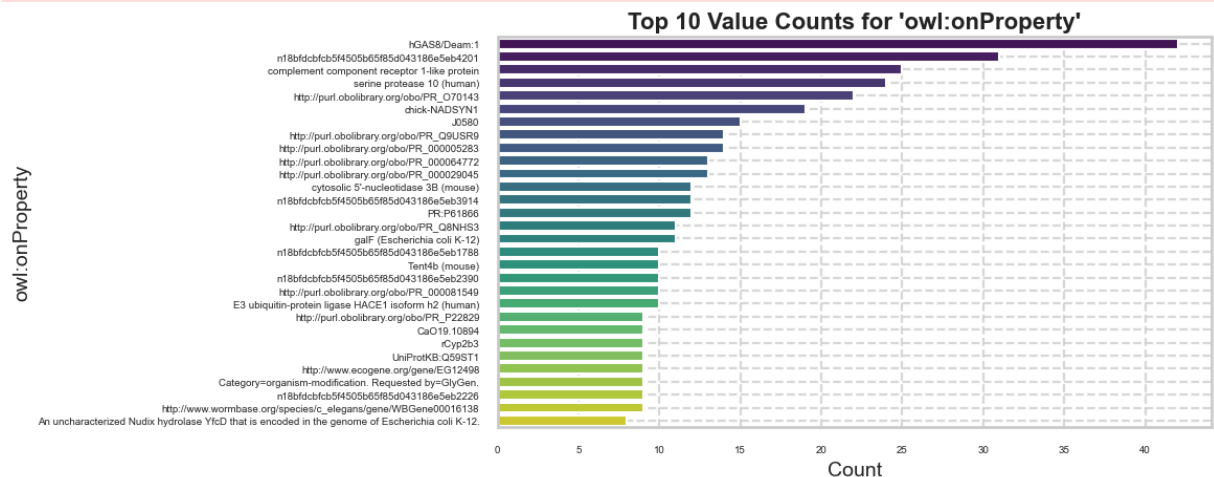
```
sns.barplot(
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.py:18: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

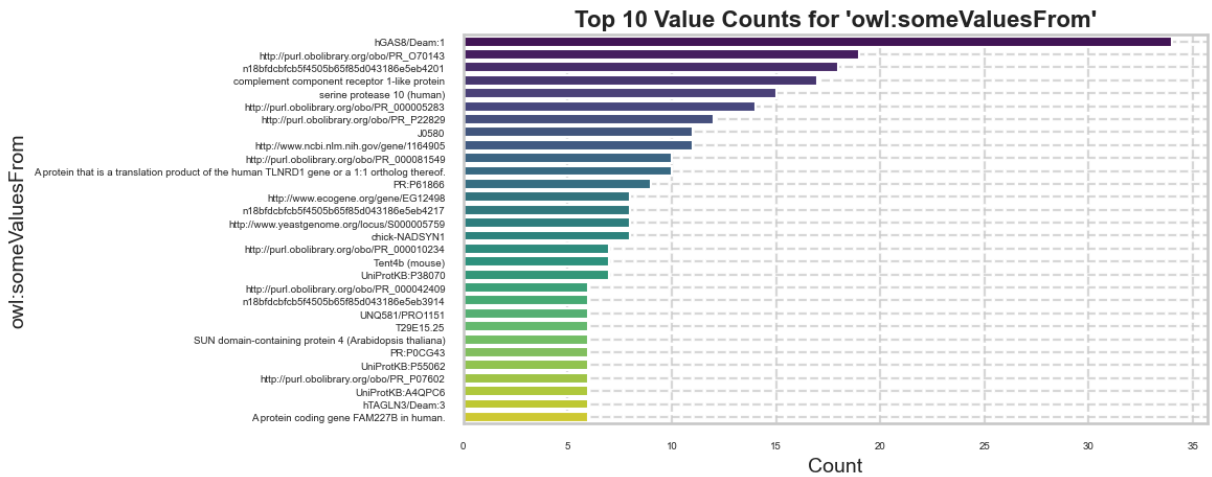
```
sns.barplot(
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.py:18: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

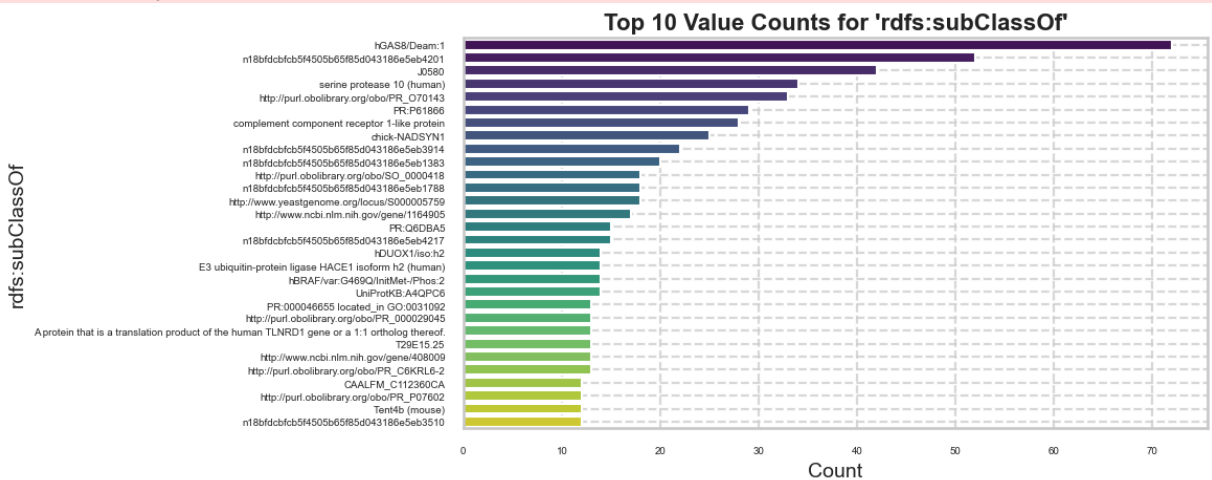
```
sns.barplot(
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.py:18: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

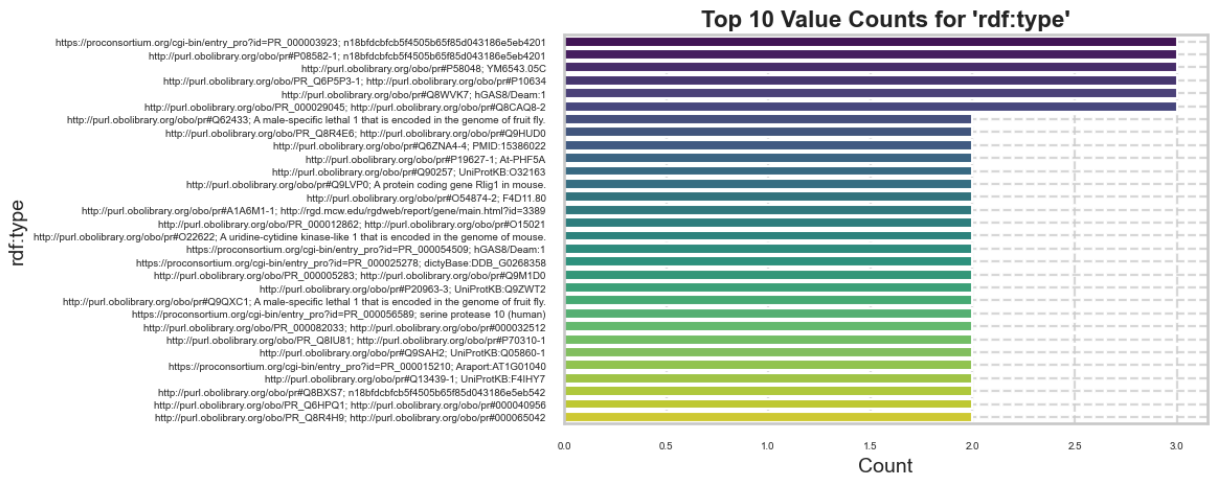
```
sns.barplot(
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3092666243.py:18: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



```
In [23]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os

# Set Seaborn theme and context
sns.set_theme(style="whitegrid")
sns.set_context("talk") # Larger fonts

# Load CSV
df = pd.read_csv("protein_VAE_summary.csv")

# Create output directory
output_dir = "plots"
os.makedirs(output_dir, exist_ok=True)

# Function to plot and save distributions
def plot_distributions(df):
    for col in df.columns:
        plt.figure(figsize=(12, 5))
        if df[col].dtype == 'object':
            top_values = df[col].value_counts().nlargest(30)
            sns.barplot(
                x=top_values.values,
                y=top_values.index,
                palette="viridis"
            )
            plt.title(f"Top 30 Value Counts for '{col}'", fontsize=16, weight='bold')
            plt.xlabel("Count", fontsize=14)
            plt.ylabel(col, fontsize=14)
        else:
            sns.histplot(
                df[col].dropna(),
                kde=True,
                bins=30,
                color="skyblue",
                edgecolor="black"
            )
            plt.title(f"Distribution of '{col}'", fontsize=16, weight='bold')
            plt.xlabel(col, fontsize=14)
            plt.ylabel("Frequency", fontsize=14)
```

```

plt.xticks(fontsize=7)
plt.yticks(fontsize=7)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()

# Save figure
safe_col_name = "".join(c if c.isalnum() else "_" for c in col)
filename = os.path.join(output_dir, f"{safe_col_name}.png")
plt.savefig(filename, dpi=300, bbox_inches='tight')

plt.show()

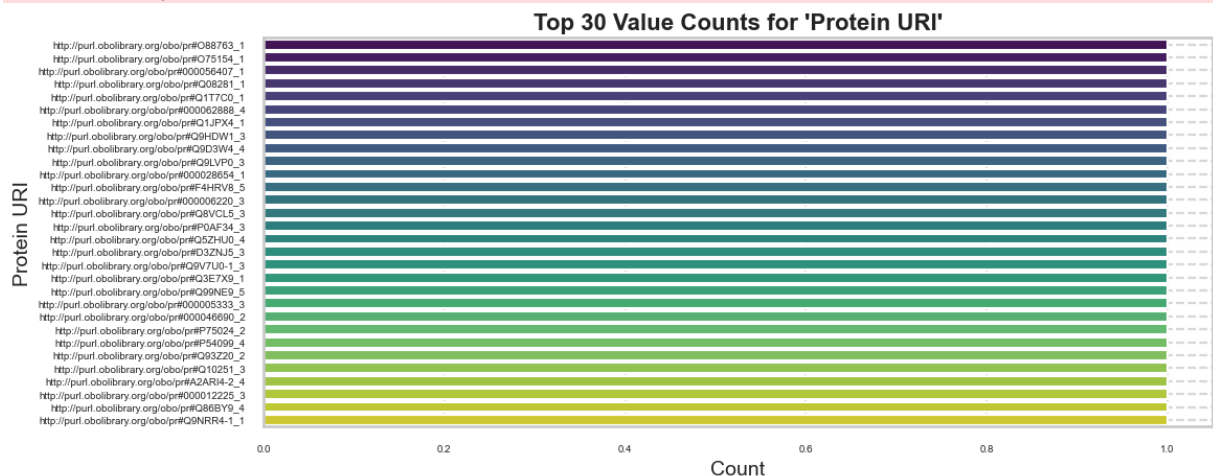
plot_distributions(df)

```

/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.
py:23: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

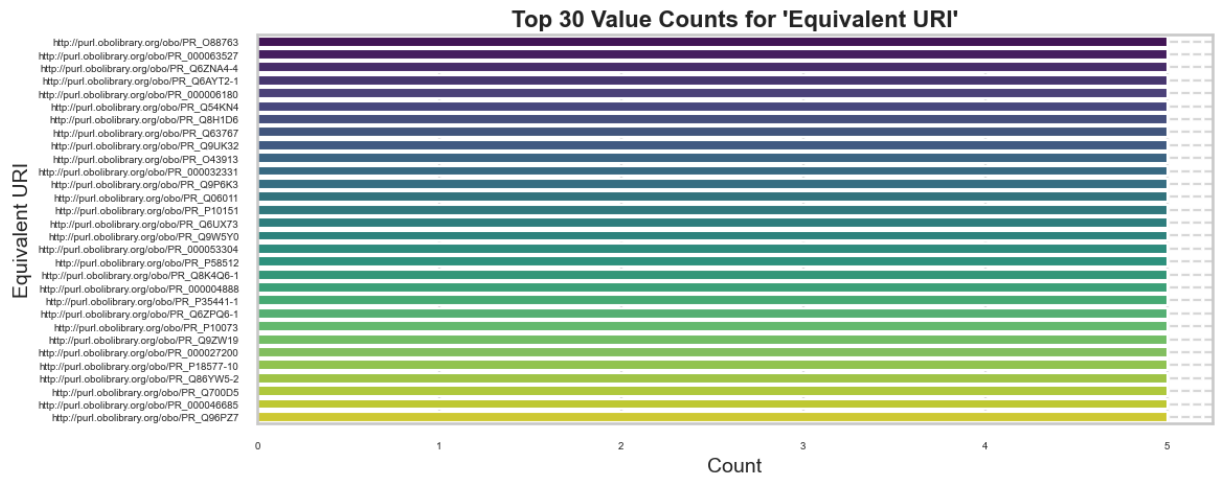
sns.barplot(



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.
py:23: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

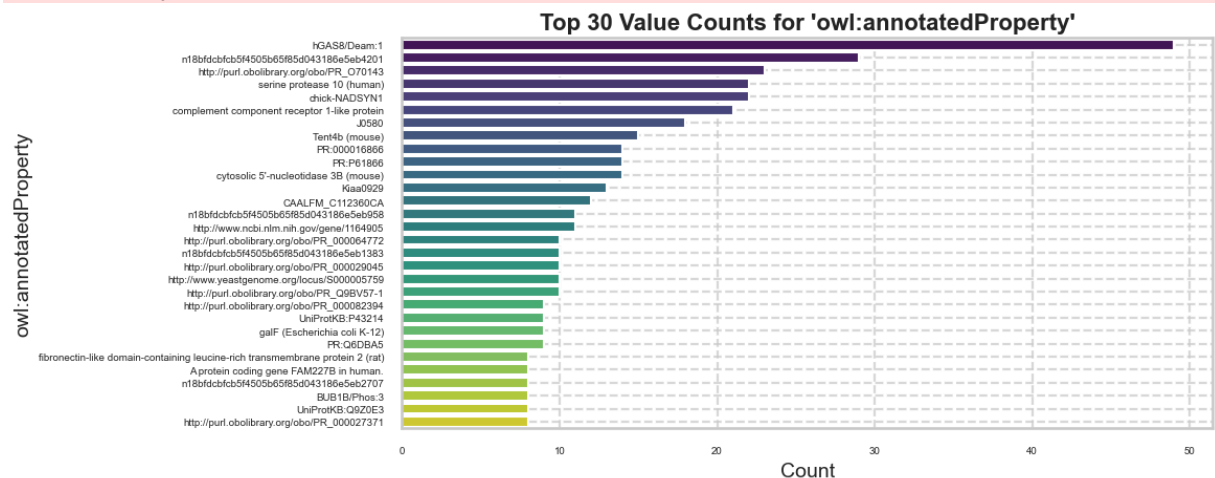
sns.barplot(



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.
py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

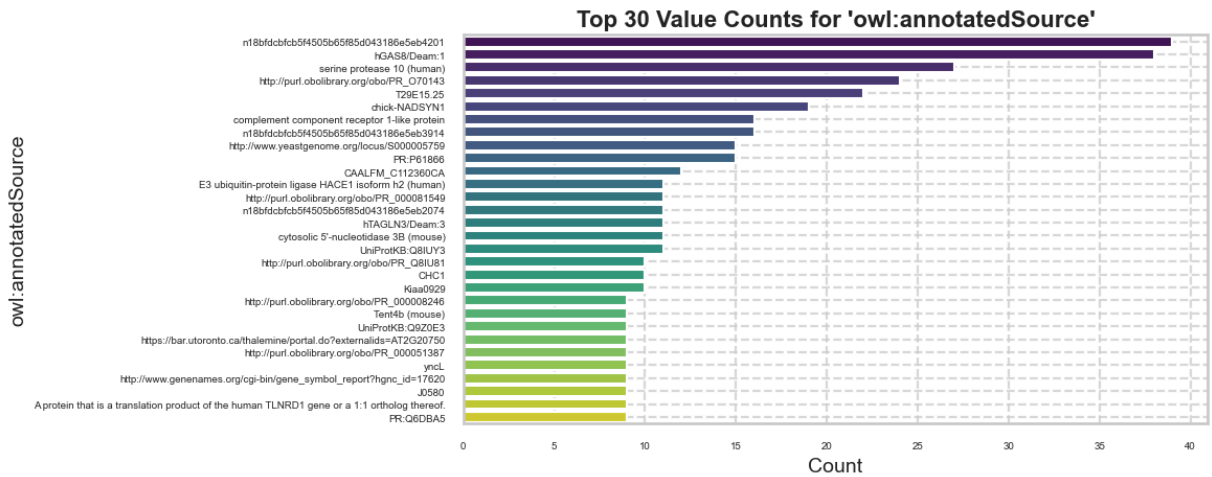
```
sns.barplot(
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.
py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.
py:23: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

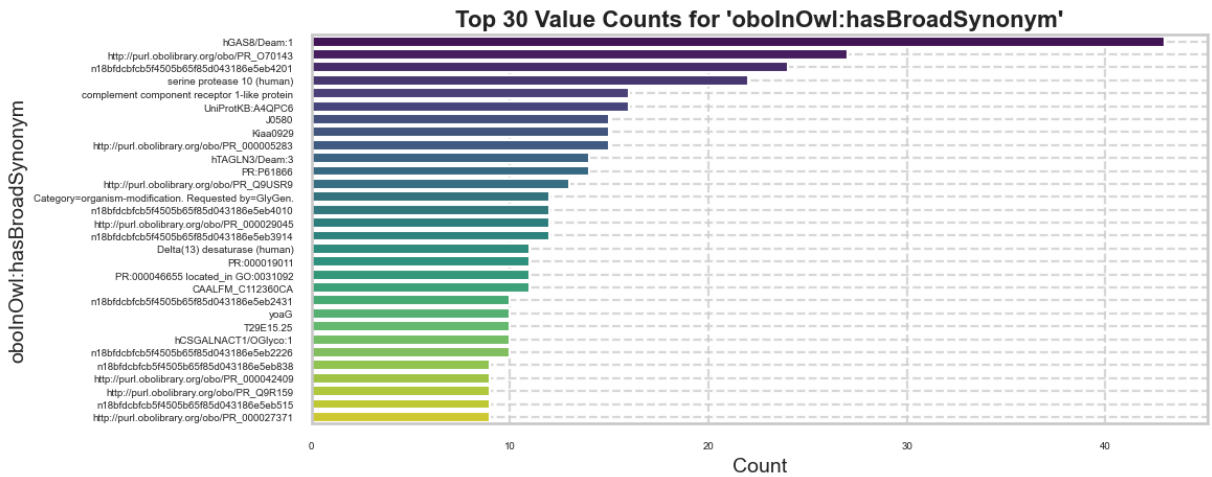
sns.barplot(



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.
py:23: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

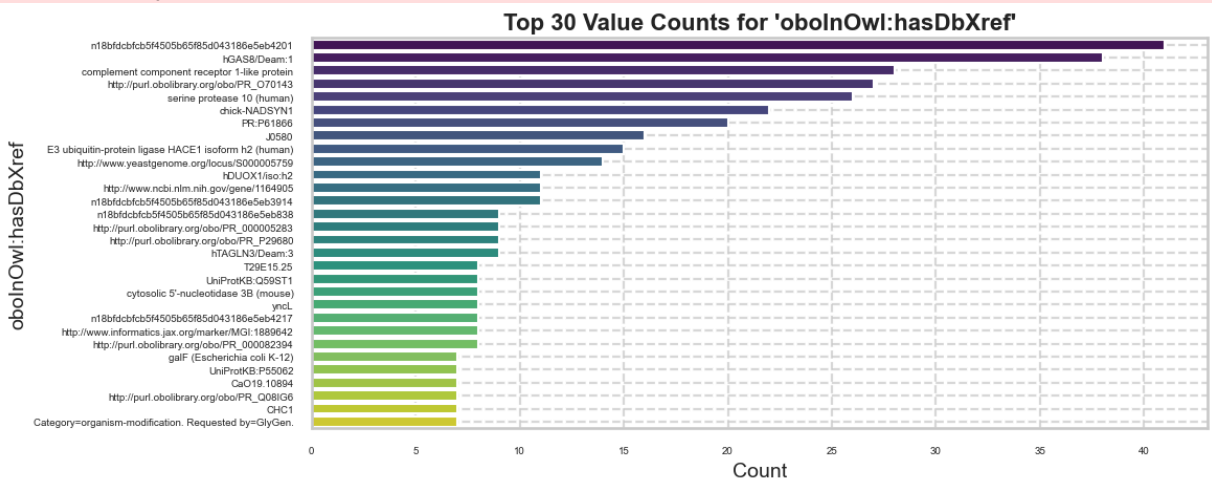
sns.barplot(



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.
py:23: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

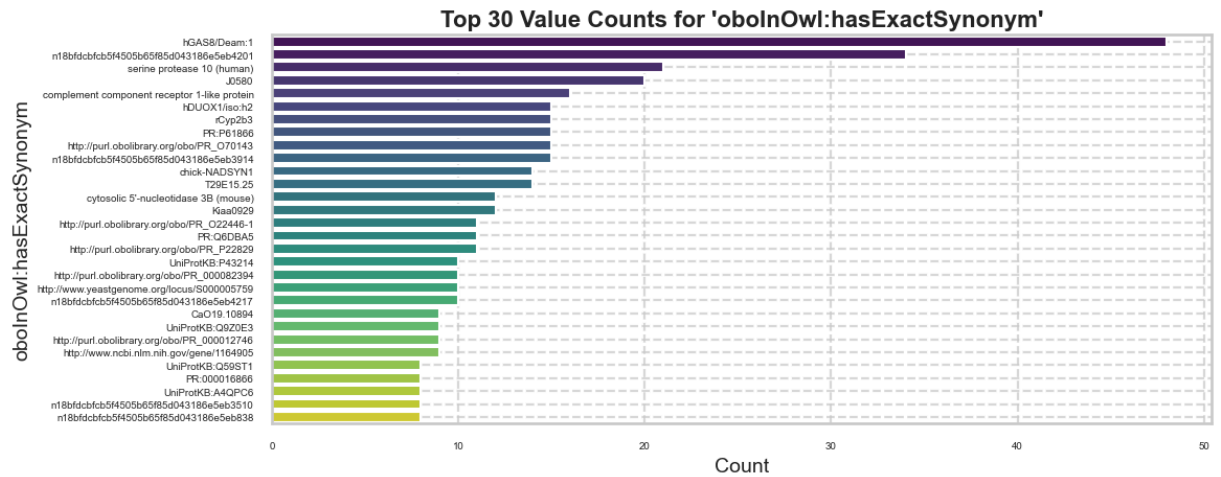
sns.barplot(



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.
py:23: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

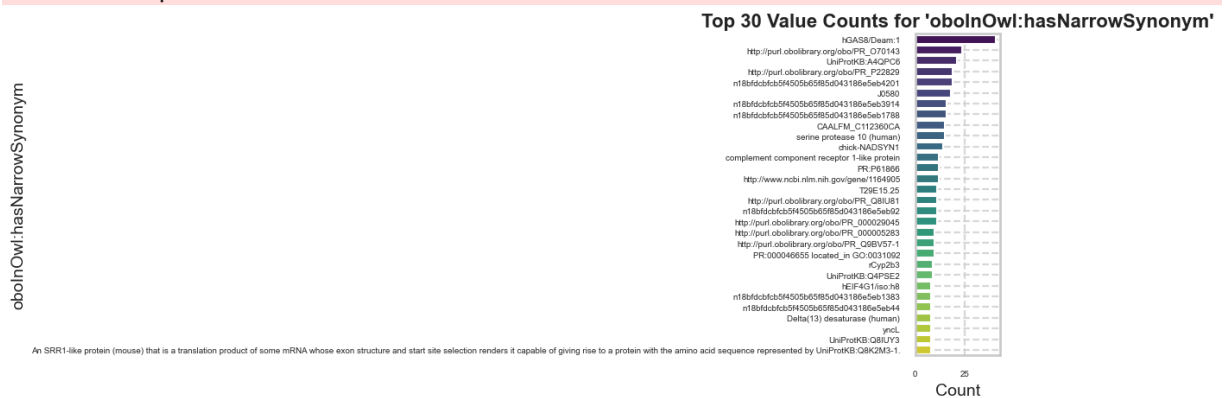
sns.barplot(



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```

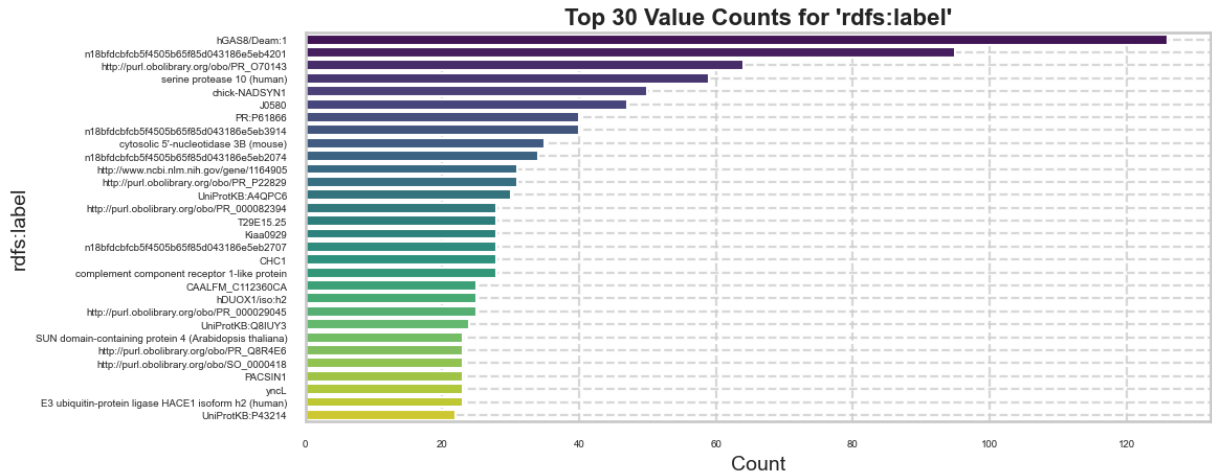
```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.py:46: UserWarning: Tight layout not applied. The left and right margins can not be made large enough to accommodate all Axes decorations.  
plt.tight_layout()
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

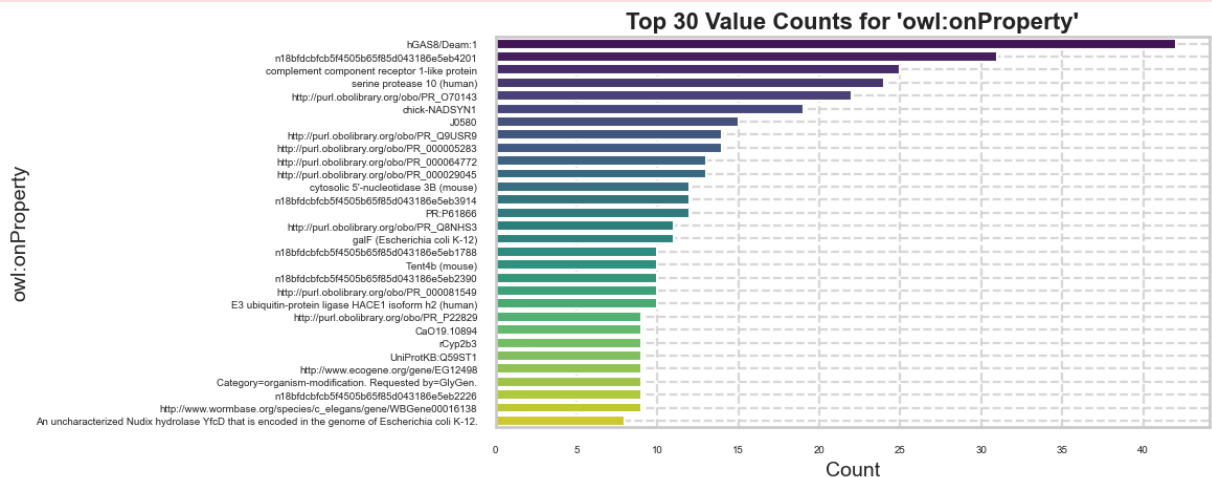
```
sns.barplot(
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

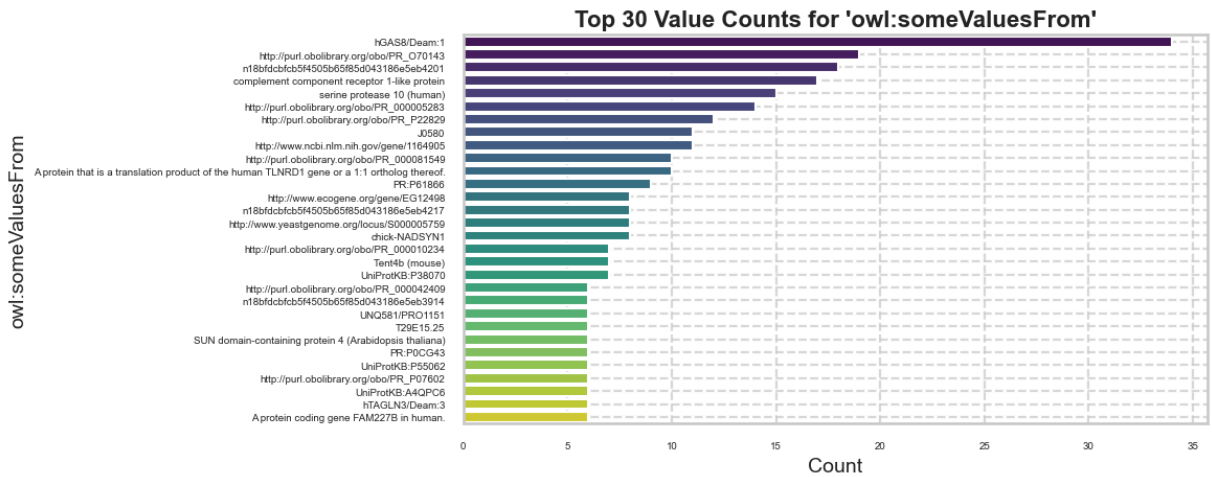
```
sns.barplot(
```



```
/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.py:23: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

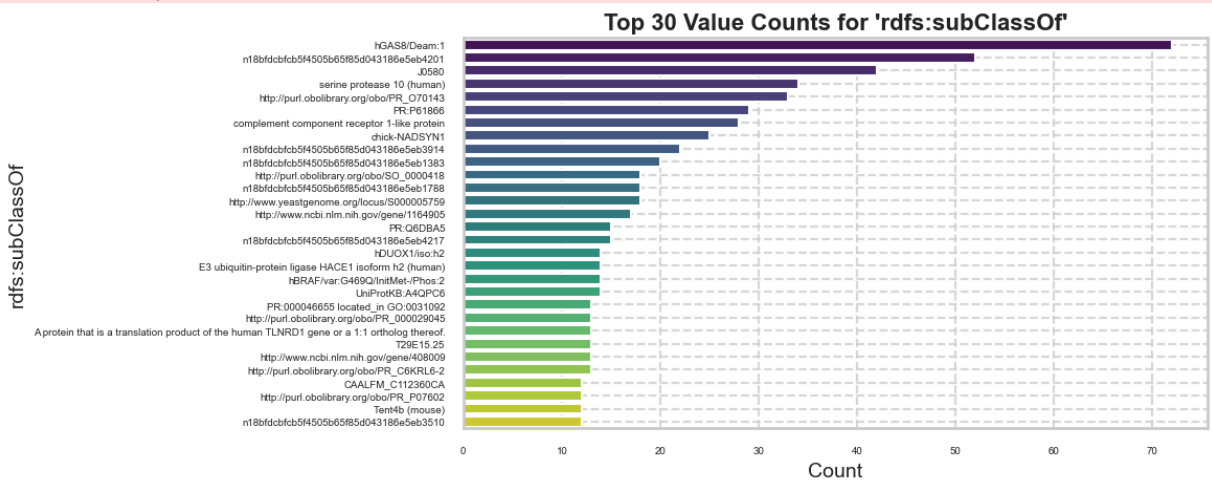
```
sns.barplot(
```



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.
py:23: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

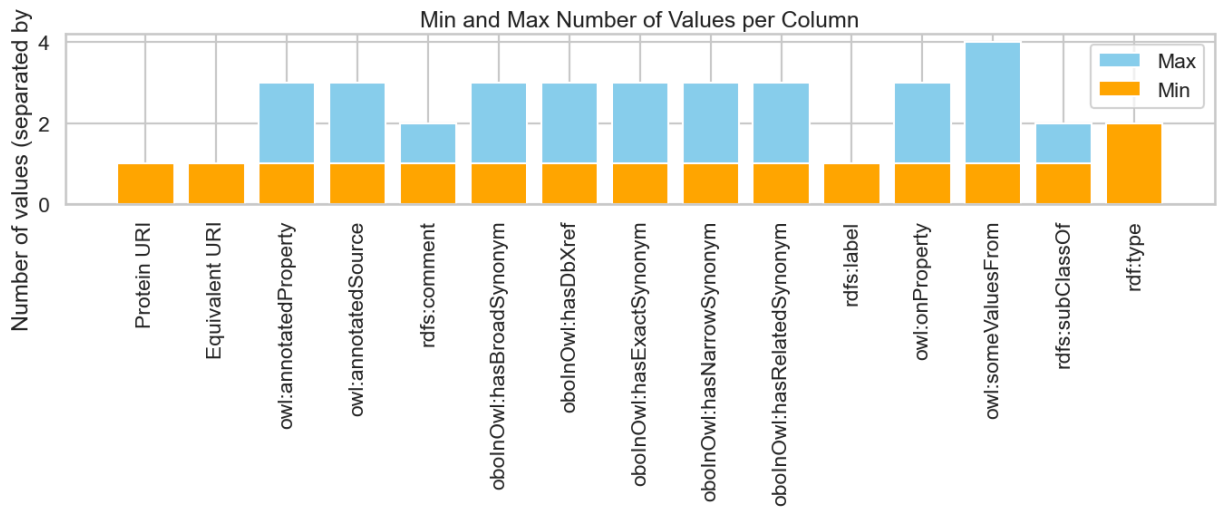
sns.barplot(



/var/folders/by/r1y5wccx25jfdv7nvpnb9pf40000gn/T/ipykernel_56106/3083737015.
py:23: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

sns.barplot(

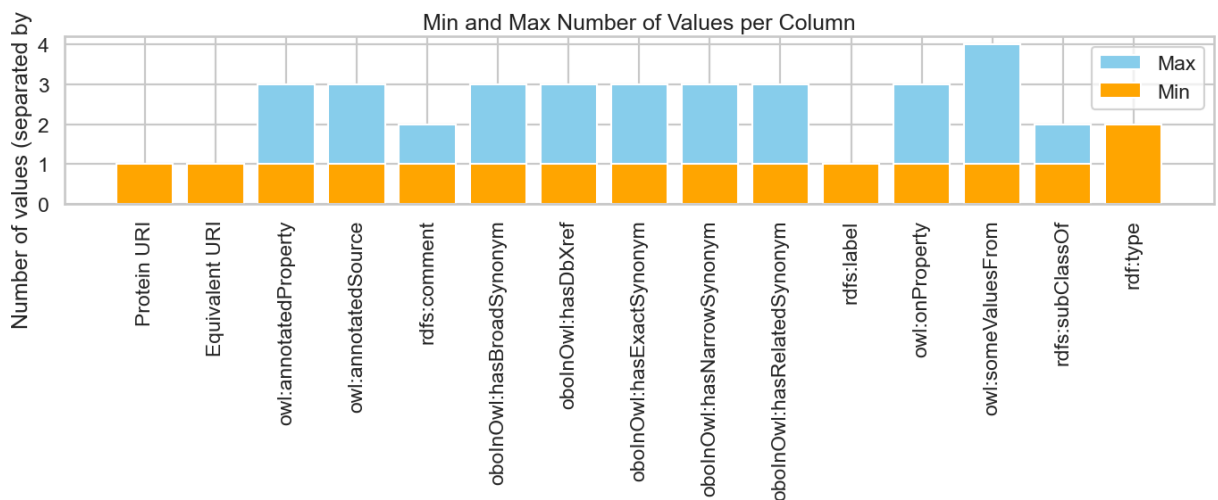


```
In [29]: import matplotlib.ticker as ticker

plt.figure(figsize=(14, 6))
plt.bar(count_df['Column'], count_df['Max'], label='Max', color='skyblue')
plt.bar(count_df['Column'], count_df['Min'], label='Min', color='orange')
plt.xticks(rotation=90)
plt.ylabel("Number of values (separated by ;) per cell")
plt.title("Min and Max Number of Values per Column")
plt.legend()

# Set y-axis ticks to integers with step 1
plt.gca().yaxis.set_major_locator(ticker.MaxNLocator(integer=True))

plt.tight_layout()
plt.show()
```



```
In [36]: import matplotlib.ticker as ticker
import numpy as np

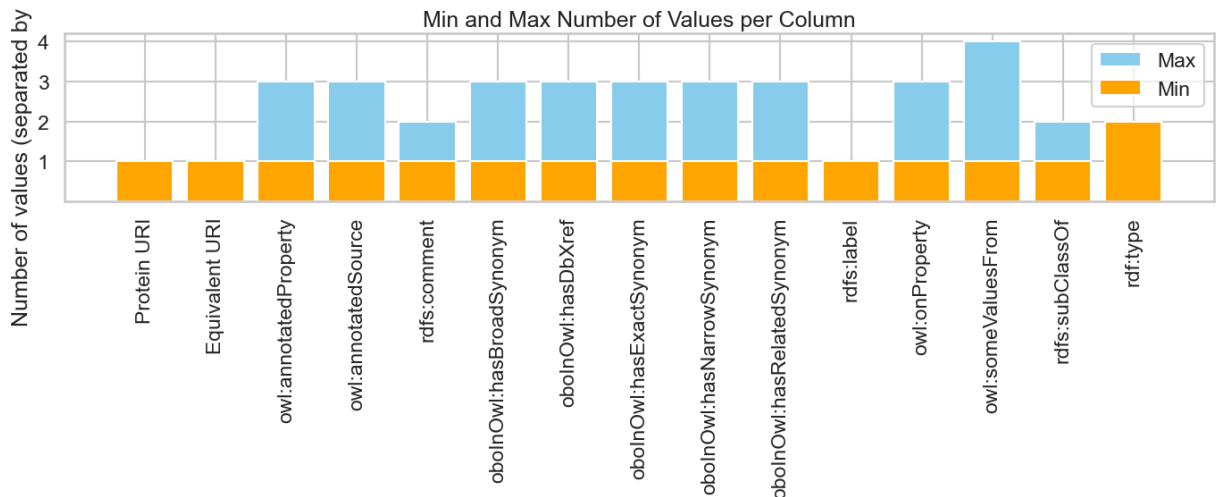
plt.figure(figsize=(14, 6))
plt.bar(count_df['Column'], count_df['Max'], label='Max', color='skyblue')
plt.bar(count_df['Column'], count_df['Min'], label='Min', color='orange')
plt.xticks(rotation=90)
plt.ylabel("Number of values (separated by ;) per cell")
```

```
plt.title("Min and Max Number of Values per Column")
plt.legend()

# Get unique y values from Min and Max columns combined
unique_ys = np.unique(np.concatenate([count_df['Min'].values, count_df['Max'].values]))

# Set y-axis ticks explicitly to those unique values
plt.gca().set_yticks(unique_ys)

plt.tight_layout()
plt.show()
```



```
In [37]: import matplotlib.ticker as ticker
import numpy as np

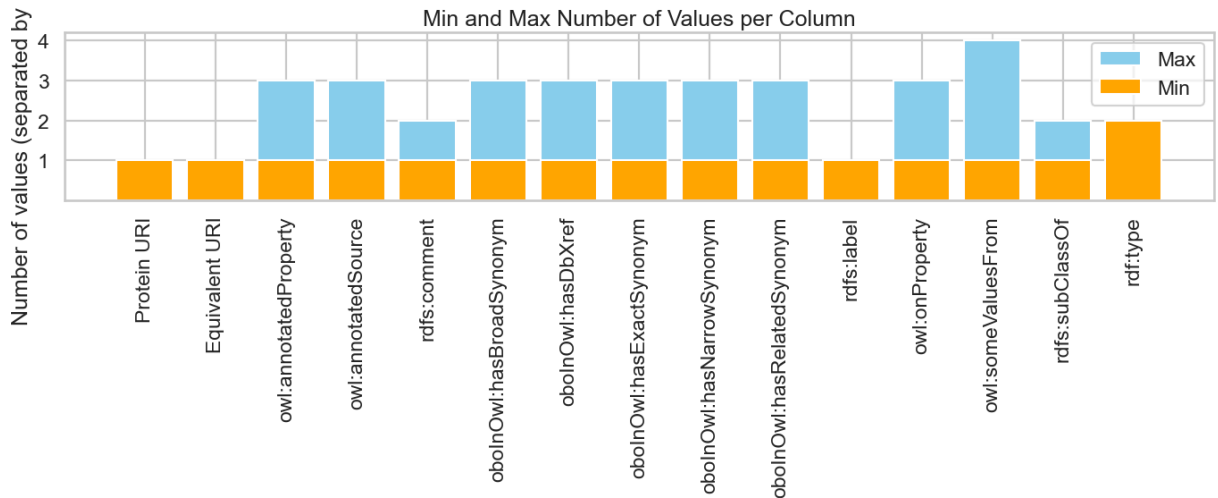
plt.figure(figsize=(14, 6))
plt.bar(count_df['Column'], count_df['Max'], label='Max', color='skyblue')
plt.bar(count_df['Column'], count_df['Min'], label='Min', color='orange')
plt.xticks(rotation=90)
plt.ylabel("Number of values (separated by ;) per cell")
plt.title("Min and Max Number of Values per Column")
plt.legend()

# Collect all integers between Min and Max inclusive for each row
all_ys = []
for min_val, max_val in zip(count_df['Min'], count_df['Max']):
    all_ys.extend(range(int(min_val), int(max_val) + 1))

# Get unique sorted values
unique_ys = np.unique(all_ys)

# Set y-axis ticks explicitly to those values
plt.gca().set_yticks(unique_ys)

plt.tight_layout()
plt.show()
```

```
In [43]: import matplotlib.ticker as ticker
import os
import matplotlib.pyplot as plt

# Make sure your count_df is ready here

# Create output directory for plots (optional)
output_dir = "plots"
os.makedirs(output_dir, exist_ok=True)

plt.figure(figsize=(14, 6))
plt.bar(count_df['Column'], count_df['Max'], label='Max', color='skyblue')
plt.bar(count_df['Column'], count_df['Min'], label='Min', color='orange')
plt.xticks(rotation=90)
plt.ylabel("Number of values (separated by ;) per cell")
plt.title("Min and Max Number of Values per Column")
plt.legend()

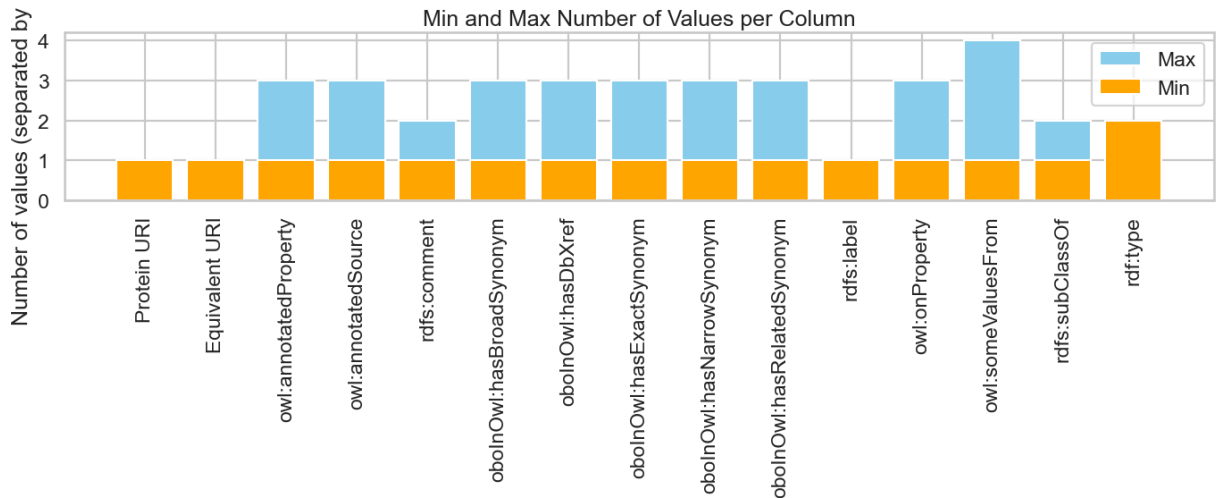
# Force y-axis to show integer ticks
plt.gca().yaxis.set_major_locator(ticker.MaxNLocator(integer=True))

plt.tight_layout()

# Save figure as PNG file
filename = os.path.join(output_dir, "min_max_values_per_column_VAE_.png")
plt.savefig(filename, dpi=300, bbox_inches='tight')

plt.show()

print(f"Plot saved to {filename}")
```



Plot saved to plots/min_max_values_per_column_VAE_.png

```
In [46]: import pandas as pd
import requests
from urllib.parse import urlparse

# Function to check if a string is a valid URI (simple check)
def is_valid_uri(uri):
    try:
        result = urlparse(uri)
        return all([result.scheme in ['http', 'https'], result.netloc])
    except:
        return False

# Function to check if URI is reachable (HTTP status 200)
def check_uri_reachable(uri):
    try:
        response = requests.head(uri, allow_redirects=True, timeout=5)
        # Accept 2xx or 3xx status codes as valid
        return response.status_code < 400
    except requests.RequestException:
        return False

# Load CSV
df = pd.read_csv("protein_VAE_summary.csv")

# Columns to check except 'Protein URI'
columns_to_check = [col for col in df.columns if col != "Protein URI"]

# Dictionary to store validation results
validation_results = {}

for col in columns_to_check:
    validation_results[col] = []

    for cell in df[col].fillna(''): # Handle NaN
        # Some cells might have multiple values separated by e.g. comma, so
        # Here we guess comma or semicolon; adapt if needed
        values = [v.strip() for v in str(cell).split(',') if ',' in str(cell)]

        # Filter values that look like URIs
```

```

uris = [v for v in values if is_valid_uri(v)]

# Validate each URI
validities = [check_uri_reachable(uri) for uri in uris]

# Append validation results for this cell (True if all URIs valid, e
# If no URIs in this cell, skip validation (assume True)
validation_results[col].append(all(validities) if validities else Tr

# Add results to dataframe as new columns with suffix "_uri_valid"
for col in columns_to_check:
    df[f"{col}_uri_valid"] = validation_results[col]

# Save results or review
df.to_csv("VAE_file_with_uri_validation.csv", index=False)
print("Validation completed and saved to your_file_with_uri_validation.csv")

```

Validation completed and saved to your_file_with_uri_validation.csv